

Universidad del Valle de Guatemala

Facultad de Ingeniería

Programación Orientada a Objetos

Segundo Ciclo, 2026

Catedrático: Erick Marroquin

Laboratorio No. 2

Festival universitario de música

Nombre: Raul Escobar

Carné: 26429

Sección # 30

Fecha de entrega: 07/09/2026

1. Análisis

Requisitos Funcionales

1. Solicitar al iniciar el nombre, código y coordinador del festival. Permitir crear otro festival que reemplace al actual y comience con ambas colecciones vacías.
2. Administrar como máximo 5 escenarios mediante un arreglo básico de objetos Escenario; sus posiciones inicialmente contienen null.
3. Guardar código, nombre, ubicación, capacidad máxima y estado de cada escenario.
4. Exigir capacidad mayor que 0 al crear o modificar un escenario; en caso contrario generar `IllegalArgumentException`.
5. Configurar un escenario en la posición indicada por el usuario, verificando límites, disponibilidad e información válida.
6. Comprobar que una posición no contenga null antes de acceder a los métodos del escenario.
7. Consultar todos los escenarios configurados e indicar sus posiciones, omitiendo las posiciones vacías.
8. Consultar un escenario por posición e informar si la posición es inválida o está vacía.
9. Modificar la capacidad y el estado de un escenario existente; rechazar posiciones vacías y capacidades inválidas.
10. Retirar un escenario configurado y asignar null a su posición.
11. Administrar una cantidad variable de artistas mediante `ArrayList<Artista>`, utilizando `ArrayList` directamente, sin `List`.
12. Guardar código, nombre artístico, género, duración en minutos y asistentes estimados de cada artista.
13. Exigir duración mayor que 0 y asistentes estimados mayores o iguales que 0; generar `IllegalArgumentException` si no se cumplen estas reglas.
14. Registrar artistas sin permitir códigos duplicados.
15. Consultar todos los artistas e informar cuando no haya registros.
16. Buscar un artista por código recorriendo el `ArrayList`.
17. Modificar los datos de un artista existente y aplicar las mismas validaciones de creación; si cambia el código, verificar su unicidad.
18. Cancelar la participación de un artista localizándolo por código y eliminándolo del `ArrayList`.
19. Mostrar cantidad de escenarios configurados, espacios disponibles y escenario de mayor capacidad.
20. Mostrar cantidad de artistas, artista de mayor duración, artista con más asistentes estimados y promedio de duración.
21. Considerar únicamente artistas registrados; evitar máximos y promedios cuando no existan elementos.
22. Manejar `InputMismatchException` al leer números mediante `Scanner`, limpiar la entrada incorrecta y permitir continuar.
23. Usar try-catch para manejar errores de entrada y validación, y al menos un finally para ejecutar una acción final tanto si hay éxito como si ocurre un error.

24. Mantener el menú de 13 opciones indicado en el enunciado y continuar después de errores manejados hasta que se seleccione Salir.

Tabla de Propiedades

<i>Clase</i>	<i>Propiedad</i>	<i>Tipo de dato</i>	<i>Visibilidad</i>	<i>Propósito</i>
<i>Festival</i>	nombre	String	private	Nombre del festival.
<i>Festival</i>	codigo	String	private	Identificación del festival.
<i>Festival</i>	coordinador	String	private	Nombre del responsable.
<i>Festival</i>	escenarios	Escenario[]	private	Arreglo de tamaño 5; conserva escenarios o null en posiciones disponibles.
<i>Festival</i>	artistas	ArrayList<Artista>	private	Colección dinámica de artistas registrados.
<i>Escenario</i>	codigo	String	private	Identificación del escenario.
<i>Escenario</i>	nombre	String	private	Nombre del escenario.
<i>Escenario</i>	ubicacion	String	private	Lugar donde se encuentra.
<i>Escenario</i>	capacidadMaxima	int	private	Máximo de asistentes permitido; debe ser mayor que 0.
<i>Escenario</i>	estado	String	private	Estado del escenario, por ejemplo disponible o cerrado.
<i>Artista</i>	codigo	String	private	Identificador único dentro del festival.
<i>Artista</i>	nombreArtistico	String	private	Nombre con el que se presenta.
<i>Artista</i>	generoMusical	String	private	Género musical de su presentación.
<i>Artista</i>	duracionMinutos	int	private	Duración de la presentación; debe ser mayor que 0.
<i>Artista</i>	asistentesEstimados	int	private	Público esperado; no puede ser negativo.
<i>Main</i>	—	—	—	No requiere atributos: Scanner y Festival serán variables locales de main.

Tabla de Métodos

Clase Escenario

<i>Método</i>	<i>Retorno</i>	<i>Visibilidad</i>	<i>Parámetros</i>	<i>Propósito</i>
Escenario	—	public	String codigo, String nombre, String ubicacion, int capacidadMaxima, String estado	Inicializa los datos; rechaza capacidad ≤ 0 con IllegalArgumentException.
getCodigo	String	public	Ninguno	Devuelve el código.
getNombre	String	public	Ninguno	Devuelve el nombre.
getUbicacion	String	public	Ninguno	Devuelve la ubicación.
getCapacidadMaxima	int	public	Ninguno	Devuelve la capacidad máxima.
getEstado	String	public	Ninguno	Devuelve el estado.
actualizarDatos	void	public	int capacidadMaxima, String estado	Valida primero la capacidad (> 0) y luego actualiza ambos datos; si falla, no modifica el objeto.
toString	String	public	Ninguno	Devuelve los datos del escenario para mostrarlos.

Clase Artista

<i>Método</i>	<i>Retorno</i>	<i>Visibilidad</i>	<i>Parámetros</i>	<i>Propósito</i>
Artista	—	public	String codigo, String nombreArtistico, String generoMusical, int duracionMinutos, int asistentesEstimados	Inicializa los datos; genera IllegalArgumentException por duración ≤ 0 o asistentes < 0 .
getCodigo	String	public	Ninguno	Devuelve el código.
getNombreArtistico	String	public	Ninguno	Devuelve el nombre artístico.
getGeneroMusical	String	public	Ninguno	Devuelve el género musical.
getDuracionMinutos	int	public	Ninguno	Devuelve la duración.
getAsistentesEstimados	int	public	Ninguno	Devuelve el público estimado.
actualizarDatos	void	public	String codigo, String nombreArtistico, String generoMusical, int duracionMinutos, int asistentesEstimados	Valida los valores numéricos antes de asignarlos. Festival verifica antes la unicidad del código.
toString	String	public	Ninguno	Devuelve los datos del artista para mostrarlos.

Clase Festival: administración de escenarios

<i>Método</i>	<i>Retorno</i>	<i>Visibilidad</i>	<i>Parámetros</i>	<i>Propósito</i>
<i>Festival</i>	—	public	String nombre, String codigo, String coordinador	Inicializa los datos, new Escenario[5] y new ArrayList<Artista>().
<i>configurarEscenario</i>	void	public	int posicion, Escenario escenario	Valida índice, objeto no nulo y posición libre; guarda el escenario. Rechaza ocupación con IllegalStateException.
<i>consultarEscenarios</i>	String	public	Ninguno	Recorre el arreglo y devuelve datos e índices de los escenarios no nulos, o un mensaje si no hay.
<i>consultarEscenario</i>	Escenario	public	int posicion	Valida índice y devuelve el objeto o null si la posición está vacía.
<i>modificarEscenario</i>	void	public	int posicion, int capacidadMaxima, String estado	Exige escenario existente y delega la validación y actualización de sus datos.
<i>retirarEscenario</i>	void	public	int posicion	Exige escenario existente y coloca null en su posición.
<i>validarPosicion</i>	void	private	int posicion	Genera IndexOutOfBoundsException si el índice no está entre 0 y 4.
<i>exigirEscenario</i>	Escenario	private	int posicion	Valida índice y genera IllegalStateException si la posición contiene null.

Clase Festival: administración de artistas

<i>Método</i>	<i>Retorno</i>	<i>Visibilidad</i>	<i>Parámetros</i>	<i>Propósito</i>
<i>registrarArtista</i>	void	public	Artista artista	Rechaza null y códigos repetidos con IllegalArgumentException; agrega el artista con add.
<i>consultarArtistas</i>	String	public	Ninguno	Recorre la colección y devuelve los datos; informa si está vacía.
<i>buscarIndiceArtista</i>	int	private	String codigo	Recorre por índices y compara códigos con equals; devuelve el índice o -1.
<i>buscarArtista</i>	Artista	public	String codigo	Devuelve el artista encontrado o null si no existe.
<i>modificarArtista</i>	void	public	String codigoActual, String nuevoCodigo, String nombreArtistico, String generoMusical, int duracionMinutos, int asistentesEstimados	Localiza el registro, verifica que el nuevo código no pertenezca a otro artista y actualiza. Genera IllegalArgumentException si no existe o se duplica el código.

<i>cancelarParticipacion</i>	boolean	public	String codigo	Busca el índice y elimina con remove(indice); devuelve true si lo eliminó y false si no existe.
------------------------------	---------	--------	---------------	---

Clase Festival: consultas y reporte

<i>Método</i>	<i>Retorno</i>	<i>Visibilidad</i>	<i>Parámetros</i>	<i>Propósito</i>
<i>getNombre</i>	String	public	Ninguno	Devuelve el nombre.
<i>getCodigo</i>	String	public	Ninguno	Devuelve el código.
<i>getCoordinador</i>	String	public	Ninguno	Devuelve el coordinador.
<i>contarEscenarios</i>	int	public	Ninguno	Cuenta las posiciones diferentes de null.
<i>contarEspaciosLibres</i>	int	public	Ninguno	Devuelve 5 menos la cantidad de escenarios configurados.
<i>escenarioMasGrande</i>	Escenario	public	Ninguno	Recorre los escenarios no nulos; devuelve el de mayor capacidad o null si no hay.
<i>contarArtistas</i>	int	public	Ninguno	Devuelve artistas.size().
<i>artistaMayorDuracion</i>	Artista	public	Ninguno	Devuelve el artista de mayor duración o null si no hay.
<i>artistaMayorAsistencia</i>	Artista	public	Ninguno	Devuelve el artista con más asistentes estimados o null si no hay.
<i>promedioDuracion</i>	double	public	Ninguno	Suma duraciones en double y divide entre size(); si está vacío genera IllegalStateException. Main evita invocarlo en ese caso.

Clase Main

<i>Método</i>	<i>Retorno</i>	<i>Visibilidad</i>	<i>Parámetros</i>	<i>Propósito</i>
<i>main</i>	void	public	String[] args	Método public static: crea Scanner y Festival, repite el menú, ejecuta operaciones y captura errores.
<i>mostrarMenu</i>	void	private	Ninguno	Método private static: imprime las 13 opciones del enunciado.
<i>leerFestival</i>	Festival	private	Scanner entrada	Método private static: solicita los datos y devuelve un festival nuevo.
<i>mostrarReporte</i>	void	private	Festival festival	Método private static: presenta los siete indicadores; si faltan escenarios o artistas informa que no hay datos.

Respuestas a las preguntas de análisis

1. ¿Qué propiedades y métodos tendrá cada clase?

Se detallan en las tablas anteriores. Festival administra el evento y sus colecciones; Escenario y Artista representan los objetos registrados; Main organiza la interacción. Cada método incluye su propósito y los datos que necesita.

2. ¿Qué tipo deben tener las propiedades y métodos?

Los textos y códigos serán String; capacidades, duraciones, asistentes y cantidades serán int. Se utilizarán Escenario[], ArrayList<Artista> y retornos Escenario o Artista para objetos. El promedio será double, la cancelación devolverá boolean y las operaciones sin resultado serán void. Los constructores no declaran retorno.

3. ¿Qué propiedad utilizará un arreglo básico?

Festival.escenarios será Escenario[], creado con new Escenario[5]. Sus índices válidos serán 0 a 4. Cada posición almacenará un Escenario o null. El menú indicará este intervalo para evitar confusión.

4. ¿Qué propiedad utilizará ArrayList?

Festival.artistas será ArrayList<Artista>, inicializado con new ArrayList<Artista>(). Almacenará únicamente artistas registrados y cambiará de tamaño al agregar o cancelar participantes. No se utilizará List.

5. ¿Qué modificadores de visibilidad se utilizarán?

Los atributos serán private para encapsular el estado. Los constructores y operaciones que utilizan otras clases serán public. Los auxiliares de búsqueda y validación de Festival serán private. Main.main será public static y sus auxiliares private static.

6. ¿Qué parámetros requerirán los métodos?

Las tablas especifican cada parámetro y su tipo. Los constructores reciben los datos iniciales; las operaciones de escenarios reciben un índice y, cuando corresponde, nuevos datos; las de artistas reciben un código y sus datos nuevos. Los cálculos operan sobre las colecciones internas y no requieren parámetros.

7. ¿Cómo se inicializarán los objetos y qué se validará?

Los constructores asignarán los valores ingresados. Festival iniciará con cinco posiciones null y un ArrayList vacío. Se validará capacidad > 0, duración > 0, asistentes ≥ 0, índice válido, disponibilidad del espacio y código de artista único. Las modificaciones validarán todos sus valores antes de cambiar el objeto para evitar actualizaciones parciales. Un nuevo festival reemplazará al anterior solo cuando su creación sea correcta.

8. ¿Cómo se reconocerá una posición vacía?

Primero se comprobará que posicion ≥ 0 y posicion < escenarios.length. Después se evaluará escenarios[posicion] != null. Solo si es verdadero se utilizarán métodos del escenario. Al retirarlo se asignará null; al recorrer el arreglo se omitirán las posiciones vacías.

9. ¿Cómo se buscará, modificará y eliminará en ArrayList?

Se recorrerá desde el índice 0 hasta `size() - 1` comparando `get(i).getCodigo().equals(codigo)`. La búsqueda devolverá el índice o -1. Para modificar, se localizará el objeto y se validarán todos los datos y cualquier nuevo código, excluyendo de la comprobación al mismo artista. Para cancelar, se usará `remove(indice)` después de terminar la búsqueda. No se eliminará dentro de un recorrido `for-each`.

10. ¿Qué excepciones se manejarán y dónde se usarán try-catch y finally?

Dentro del ciclo del menú de Main, un try abarcará la lectura numérica y la operación seleccionada. Los catch manejarán `InputMismatchException` (texto o número fuera del rango de int), `IllegalArgumentException` (datos inválidos o código duplicado), `IndexOutOfBoundsException` (posición fuera de 0 a 4) e `IllegalStateException` (espacio ocupado, escenario inexistente o promedio sin artistas). Cada captura mostrará un mensaje y devolverá el control al menú.

El catch de `InputMismatchException` descartará la línea incorrecta con `nextLine()` antes de volver al menú. Después de una lectura numérica válida también se consumirá el salto de línea antes de solicitar texto. Un try exterior abarcará la ejecución completa del menú y su finally cerrará `Scanner` al terminar, tanto por salida normal como por un error que abandone ese bloque. Los errores manejados dentro del menú permitirán continuar sin cerrar `Scanner`. Se evitará `NullPointerException` comprobando null antes de utilizar objetos. Se comprobará `isEmpty()` antes de calcular máximos y promedios; no se dividirá entre cero. `Scanner` se cerrará al salir del programa, no después de cada operación.

2. Diseño

